

DOCUMENTO TÉCNICO · VERSIÓN 1.0

KAIROS

Especificación de requisitos y diseño técnico
Plataforma de gestión del Semillero de Machine Learning

Estado	Aprobado para implementación
Versión	1.0
Fecha	5 de septiembre de 2026
Repositorio	KAIROS
Dominio de despliegue	kairospartners.uk
Alcance	50 usuarios · 10 proyectos activos · ~1.000 tareas por semestre
Presupuesto de infraestructura	0 USD/mes (solo el dominio)
Documentos consolidados	PRD · business-rules · data-model · architecture · api-contract · user-stories · roadmap

Contenido

1. Introducción

1.1 Propósito · 1.2 Alcance · 1.3 Definiciones y acrónimos · 1.4 Documentos de referencia y precedencia

2. Descripción general

2.1 Contexto y problema · 2.2 Objetivos y métricas · 2.3 Actores · 2.4 Los dos planos de roles · 2.5 Restricciones · 2.6 Supuestos y dependencias

3. Requisitos funcionales

3.1 Autenticación y acceso · 3.2 Usuarios y roles globales · 3.3 Proyectos · 3.4 Roles y permisos de proyecto · 3.5 Tareas · 3.6 Notificaciones · 3.7 Lecciones

4. Requisitos no funcionales

5. Reglas de negocio

5.1 Catálogo de permisos · 5.2 Roles predeterminados · 5.3 Ciclo de vida de la tarea · 5.4 Propiedad implícita · 5.5 Proyectos y roles · 5.6 Notificaciones e idempotencia · 5.7 Autenticación · 5.8 Matriz de autorización

6. Arquitectura del sistema

6.1 Vista general · 6.2 C4 nivel 1: contexto · 6.3 C4 nivel 2: contenedores · 6.4 C4 nivel 3: componentes · 6.5 Estructura del proyecto · 6.6 Fronteras entre módulos · 6.7 Autenticación y autorización · 6.8 Proceso programado · 6.9 Frontend · 6.10 Manejo de errores

7. Modelo de datos

7.1 Convenciones · 7.2 Tipos enumerados · 7.3 Tablas · 7.4 Mapa entidad-relación · 7.5 Normalización · 7.6 Datos iniciales

8. Contrato de la API

9. Stack tecnológico y despliegue

10. Estrategia de pruebas

11. Plan de implementación

12. Fuera de alcance

13. Riesgos y mitigaciones

14. Decisiones de arquitectura registradas

15. Matriz de trazabilidad

Anexo A. Discrepancias detectadas en la documentación de origen

1. Introducción

1.1 Propósito

Este documento consolida en una sola pieza los requisitos, las reglas de negocio, el diseño de datos, la arquitectura y el contrato de servicio de **KAIROS**, la plataforma interna de gestión del Semillero de Machine Learning. Está dirigido al equipo de desarrollo (cuatro personas) y a quien deba evaluar o auditar el trabajo. Sustituye la lectura dispersa de los siete documentos de docs/ sin reemplazarlos: aquellos siguen siendo la fuente viva y este es su corte en la versión 1.0.

1.2 Alcance del producto

Un gestor de proyectos reducido, de uso interno, que resuelve exactamente tres problemas: nadie sabe con certeza qué tareas tiene asignadas, los vencimientos se pasan sin aviso, y el material de estudio está disperso. Para ello ofrece proyectos con tablero Kanban, tareas con responsables y fecha límite, recordatorios automáticos por correo, y un catálogo de lecciones que enlaza material alojado en GitHub.

Todo lo que no sirva directamente a esos tres problemas queda fuera (sección 12). **No es un clon de Jira**: ante la duda entre una solución simple y una general, se elige la simple.

1.3 Definiciones y acrónimos

Término	Significado
RF-xx	Requisito funcional numerado (sección 3).
RNF-xx	Requisito no funcional numerado (sección 4).
RN-xx	Regla de negocio numerada (sección 5). Es la capa normativa del sistema.
HU-xx	Historia de usuario con criterios de aceptación en Gherkin.
EB-xx	Escenario de error o caso borde.
Rol global	Rol fijo definido por el sistema: ADMIN, LESSON_EDITOR, MEMBER. Un usuario tiene exactamente uno.
Rol de proyecto	Rol dinámico con permisos configurables, propio de un único proyecto.
Permiso	Código de un catálogo cerrado de 14 elementos (ej. <code>task.review</code>) que habilita una operación dentro de un proyecto.
Entrega	<i>Submission</i> . Registro obligatorio del trabajo realizado para pasar una tarea a revisión.
Despacho	<i>Dispatch</i> . Fila en <code>notification_dispatches</code> que garantiza que un recordatorio se envía una sola vez.
Arranque en frío	Demora de la primera petición cuando el servicio gratuito de Render estaba suspendido.
C4	Modelo de documentación arquitectónica en cuatro niveles: contexto, contenedores, componentes y código.

1.4 Documentos de referencia y regla de precedencia

Documento	Contiene
docs/PRD.md	Requisitos numerados, escenarios de uso y de error, alcance.
docs/business-rules.md	Permisos, transiciones de estado y reglas. Fuente de verdad.

docs/data-model.md	Tablas, restricciones, índices, normalización.
docs/architecture.md	Módulos, autorización, proceso programado, despliegue.
docs/api-contract.md	Endpoints, esquemas, códigos de error.
docs/user-stories.md	Criterios de aceptación en Gherkin; base de las pruebas de integración.
docs/roadmap.md	Orden de implementación en ocho fases.
CLAUDE.md	Convenciones obligatorias para quien escriba código en el repositorio.

Precedencia. Ante contradicción entre documentos manda `business-rules.md`. Las discrepancias detectadas al redactar este documento están registradas en el Anexo A en lugar de resolverse en silencio.

2. Descripción general

2.1 Contexto y problema

El semillero coordina hoy sus proyectos, tareas y material de estudio por canales informales: chats, carpetas sueltas y repositorios dispersos. De ahí nacen los tres problemas del alcance. La plataforma los ataca sin pretender gobernar el proceso completo de ingeniería del equipo.

2.2 Objetivos y métricas de éxito

Métrica	Objetivo
Adopción	100 % de los miembros activos con cuenta creada en las primeras dos semanas.
Tareas vencidas sin entrega	Reducción respecto al semestre anterior.
Costo de operación	0 USD/mes en infraestructura; solo el dominio (~10 USD/año).
Tiempo de carga inicial	Según RNF-02: nunca una pantalla en blanco, aviso explícito a los 3 s, espera máxima de 90 s.

2.3 Actores del sistema

Actor	Descripción
Administrador	Coordinador del semillero. Invita usuarios, crea proyectos, asigna roles globales, configura las notificaciones y publica lecciones. Tiene lectura universal sobre los proyectos, no escritura.
Editor de Lecciones	Rol global. Mantiene el catálogo de lecciones. Ningún privilegio sobre proyectos.
Miembro	Rol global base. Todo lo que puede hacer depende de su rol dentro de cada proyecto.
Líder de proyecto	No es un rol global sino un rol dentro de un proyecto. Gestiona ese proyecto y solo ese.
Sistema (scheduler)	Proceso programado externo que dispara el envío de recordatorios mediante un endpoint interno.

2.4 Los dos planos de roles

Es el concepto estructural del que dependen casi todas las decisiones de autorización. Existen dos sistemas de roles **independientes que no se mezclan nunca**:

- **Plano global (fijo).** Definido por el sistema y no editable. Un usuario tiene exactamente uno de ADMIN, LESSON_EDITOR o MEMBER.
- **Plano de proyecto (dinámico).** Cada proyecto tiene su propio conjunto de roles con permisos configurables. Un usuario tiene exactamente un rol por cada proyecto del que sea miembro.

De ahí se derivan dos consecuencias que el código debe respetar sin excepción: el rol global ADMIN **no** otorga permisos de escritura dentro de un proyecto (RN-01), y ser líder del proyecto A no otorga absolutamente nada en el proyecto B (RN-04).

2.5 Restricciones del proyecto

Restricción	Implicación de diseño
Costo cero	Todo opera en planes gratuitos: Supabase, Render, Cloudflare Pages, GitHub Actions y Resend. Ninguna pieza de pago entra al diseño.

Servicio suspendible	Render suspende el backend tras ~15 min de inactividad y Supabase pausa el proyecto tras una semana. Ambos se mitigan con un ping programado y con manejo explícito del arranque en frío.
Sin trabajos programados	Render no ofrece cron en el plan gratuito: el disparador es externo (GitHub Actions) contra un endpoint interno.
Escala pequeña	50 usuarios y ~1.000 tareas por semestre. No se justifica ninguna optimización más allá de índices correctos ni ninguna desnormalización.
Idioma	Interfaz, documentación y mensajes de error visibles en español. Código, nombres de tablas, campos, endpoints y códigos de error en inglés.
Equipo	Cuatro personas, un semestre. El plan de implementación está ordenado para que las fases 0 a 4 ya constituyan un producto útil.

2.6 Supuestos y dependencias

- El material de estudio ya vive en GitHub y seguirá viviendo ahí: la plataforma **no aloja archivos**, solo enlaza.
- El dominio `kairospartners.uk` está disponible y puede verificarse en Resend con registros SPF, DKIM y DMARC.
- El cron de GitHub Actions puede retrasarse varios minutos e incluso saltarse una ejecución. Para avisos diarios es tolerable y RN-29 lo contempla explícitamente.
- El acceso a la plataforma es cerrado: no hay registro público y el primer administrador se crea por línea de órdenes, nunca por un endpoint expuesto.

3. Requisitos funcionales

Prioridad según MoSCoW reducido: **Debe** (obligatorio para la versión 1) y **Debería** (deseable, postergable).

3.1 Autenticación y acceso

ID	Requisito	Prioridad
RF-01	El registro es cerrado . Nadie puede crear una cuenta por su cuenta; el acceso se otorga solo por invitación de un Administrador.	Debe
RF-02	El Administrador genera una invitación indicando correo y rol global. El sistema crea un enlace con token de un solo uso y vigencia de 7 días, y lo envía por correo. El enlace también puede copiarse para compartirlo por otro canal.	Debe
RF-03	El invitado abre el enlace, define su nombre y contraseña, y con eso queda creada su cuenta. El token se invalida en el momento de usarse.	Debe
RF-04	El Administrador puede reenviar una invitación —lo cual invalida el token anterior y genera uno nuevo— y revocar una invitación pendiente.	Debe
RF-05	Inicio de sesión con correo y contraseña. Devuelve un token de acceso y un token de refresco.	Debe
RF-06	Cierre de sesión: revoca el token de refresco vigente.	Debe
RF-07	Recuperación de contraseña con token de un solo uso y vigencia de 1 hora. La respuesta del endpoint de solicitud es idéntica exista o no el correo, para no filtrar qué direcciones están registradas.	Debe
RF-08	Un usuario autenticado puede cambiar su contraseña indicando la actual, y puede editar su nombre.	Debe
RF-09	El Administrador puede desactivar un usuario: no podrá iniciar sesión, dejará de recibir notificaciones y no aparecerá en los selectores de asignación, pero su historial permanece intacto y visible. No existe eliminación física de usuarios.	Debe

3.2 Usuarios y roles globales

ID	Requisito	Prioridad
RF-10	El Administrador ve el listado de usuarios con nombre, correo, rol global, estado y fecha de ingreso, con búsqueda por nombre o correo.	Debe
RF-11	El Administrador puede cambiar el rol global de cualquier usuario.	Debe
RF-12	El sistema garantiza que siempre exista al menos un Administrador activo: el último Administrador no puede degradarse ni desactivarse a sí mismo.	Debe

3.3 Proyectos

ID	Requisito	Prioridad
RF-13	Solo el Administrador crea proyectos, indicando nombre, descripción y fecha de inicio.	Debe
RF-14	Al crear un proyecto, el Administrador designa a su primer líder. El sistema crea automáticamente los tres roles de proyecto predeterminados (Líder, Colaborador, Observador) y agrega al designado con el rol Líder.	Debe
RF-15	Quien tenga <code>member</code> . <code>add</code> puede agregar al proyecto usuarios ya existentes en la plataforma, con un rol del proyecto. Desde un proyecto no se invita gente nueva a la plataforma.	Debe
RF-16	Quien tenga <code>member</code> . <code>remove</code> puede retirar miembros. Retirar a un miembro no borra sus tareas ni entregas: la tarea queda sin ese responsable y el historial se conserva.	Debe

RF-17	Quien tenga <code>role.assign</code> puede cambiar el rol que un miembro tiene dentro del proyecto.	Debe
RF-18	Un proyecto puede archivar (<code>project.archive</code>). Un proyecto archivado es de solo lectura : se consulta completo pero no admite modificación alguna ni genera notificaciones. Puede desarchivarse.	Debe
RF-19	Los proyectos nunca se eliminan.	Debe

3.4 Roles y permisos de proyecto

ID	Requisito	Prioridad
RF-20	Cada proyecto nace con tres roles del sistema: Líder (todos los permisos), Colaborador (ver, comentar, mover y entregar sus propias tareas) y Observador (ver y comentar). No se pueden eliminar ni renombrar.	Debe
RF-21	Quien tenga <code>role.manage</code> puede crear roles personalizados dentro de su proyecto, con nombre, color y una selección libre de permisos del catálogo.	Debe
RF-22	El sistema registra qué usuario creó cada rol personalizado y cuándo, y lo muestra en la interfaz de administración de roles.	Debe
RF-23	Un rol personalizado solo existe dentro del proyecto donde fue creado. No se comparte entre proyectos.	Debe
RF-24	Un rol personalizado no puede eliminarse mientras tenga miembros asignados; primero hay que reasignarlos.	Debe

3.5 Tareas

ID	Requisito	Prioridad
RF-25	Quien tenga <code>task.create</code> crea tareas dentro de un proyecto con título, descripción, periodicidad, fecha límite y responsables.	Debe
RF-26	La periodicidad (Puntual, Semanal, Mensual, Semestral) es una etiqueta descriptiva. No genera tareas automáticamente . Sirve para filtrar y para comunicar el ritmo esperado.	Debe
RF-27	Una tarea puede tener uno o varios responsables.	Debe
RF-28	La tarea recorre cinco estados fijos: BACKLOG → TODO → IN_PROGRESS → IN_REVIEW → DONE, con las transiciones de la sección 5.3.	Debe
RF-29	El proyecto se visualiza como tablero Kanban con una columna por estado, con arrastrar y soltar en escritorio y selector de estado en móvil.	Debe
RF-30	Un responsable mueve sus propias tareas entre TODO e IN_PROGRESS sin permisos adicionales. Mover tareas ajenas requiere <code>task.change_status_any</code> .	Debe
RF-31	Para pasar una tarea a IN_REVIEW, el responsable debe registrar una entrega : descripción obligatoria y, opcionalmente, la URL del commit o pull request en GitHub.	Debe
RF-32	La URL de la entrega se valida contra el formato de enlaces de GitHub (commit, pull request o árbol de repositorio). No se suben archivos a la plataforma.	Debe
RF-33	Quien tenga <code>task.review</code> aprueba la entrega (la tarea pasa a DONE) o la devuelve con un comentario obligatorio (la tarea vuelve a IN_PROGRESS).	Debe
RF-34	Una tarea guarda todo su historial de entregas, no solo la última.	Debe
RF-35	Las tareas admiten comentarios de cualquier miembro con <code>task.comment</code> . El autor puede editar o borrar los suyos.	Debe
RF-36	El tablero se filtra por responsable, periodicidad, estado y «vencidas». Existe además una vista personal «Mis tareas» que cruza todos los proyectos del usuario.	Debe

RF-37	Las tareas se pueden eliminar solo con <code>task.delete</code> , y el borrado es lógico.	Debería
-------	---	---------

3.6 Notificaciones

ID	Requisito	Prioridad
RF-38	Toda notificación se entrega por dos canales: dentro de la aplicación (campana con historial y contador de no leídas) y por correo electrónico.	Debe
RF-39	Asignación de tarea: cuando un usuario es agregado como responsable recibe una notificación inmediata con el título de la tarea, el proyecto y la fecha límite.	Debe
RF-40	Recordatorio de vencimiento: los responsables de una tarea sin terminar reciben un aviso los días previos configurados por el Administrador.	Debe
RF-41	Tarea vencida: pasada la fecha límite sin que la tarea esté en DONE, se notifica a los responsables y, adicionalmente, a los miembros con <code>task.review</code> del proyecto (véase Anexo A.2).	Debe
RF-42	Entrega revisada: cuando una entrega es aprobada o devuelta, se notifica a quien la envió.	Debe
RF-43	La configuración de recordatorios es global , no por proyecto, y solo el Administrador la modifica: días de aviso previo (por defecto 3, 1 y 0), hora de envío (por defecto 07:00 hora de Colombia) e interruptor de avisos de tareas vencidas.	Debe
RF-44	El sistema nunca envía el mismo recordatorio dos veces al mismo usuario para la misma tarea en el mismo día, aunque el proceso programado se ejecute varias veces.	Debe
RF-45	Los proyectos archivados y los usuarios desactivados no generan ni reciben notificaciones.	Debe

3.7 Lecciones

ID	Requisito	Prioridad
RF-46	El catálogo se organiza en módulos ; cada módulo agrupa lecciones ordenadas; cada lección contiene una lista de recursos .	Debe
RF-47	Solo el Administrador y el Editor de Lecciones pueden crear, editar, publicar o despublicar módulos y lecciones. Los líderes de proyecto no tienen esta facultad.	Debe
RF-48	Un recurso tiene tipo (Notebook, PDF, Video, Repositorio, Artículo), título y URL. La plataforma no aloja archivos .	Debe
RF-49	Los módulos y lecciones tienen estado borrador o publicado. Los borradores solo son visibles para el Administrador y el Editor de Lecciones.	Debe
RF-50	El orden de módulos, lecciones y recursos lo define manualmente el editor.	Debe
RF-51	Todos los usuarios autenticados pueden consultar el catálogo publicado y buscar por título o descripción.	Debe
RF-52	No se registra progreso ni avance por usuario en las lecciones.	Debe

4. Requisitos no funcionales

ID	Atributo	Requisito
RNF-01	Diseño adaptable	La interfaz funciona desde 360 px de ancho. El tablero Kanban cambia a vista de lista con selector de estado por debajo de 768 px. Las áreas táctiles miden al menos 44 × 44 px.
RNF-02	Arranque en frío	El backend se suspende tras ~15 min de inactividad y la primera petición puede tardar cerca de un minuto. El frontend muestra un estado de carga explícito («Despertando el servidor...») tras 3 s de espera, con reintento automático y tiempo de espera de 90 s. Un ping programado cada 10 min en horario hábil reduce la frecuencia del problema.
RNF-03	Costo	Todo el sistema opera dentro de los planes gratuitos de Supabase, Render, Cloudflare/Vercel, GitHub Actions y Resend. El único gasto autorizado es el dominio.
RNF-04	Escala	50 usuarios, 10 proyectos activos y ~1.000 tareas por semestre. No se requieren optimizaciones más allá de índices correctos.
RNF-05	Seguridad	Contraseñas con Argon2id. Tokens de acceso JWT de 15 minutos; tokens de refresco de 7 días, rotativos y almacenados solo como hash. Los tokens de invitación y de restablecimiento se guardan hasheados. Toda comprobación de permisos ocurre en el backend; el frontend solo oculta lo que el usuario no puede hacer, nunca es la barrera.
RNF-06	Base de datos	Modelo en tercera forma normal. Claves primarias UUID. Marcas de tiempo en <code>timestampz</code> (UTC), presentadas al usuario en <i>América/Bogotá</i> .
RNF-07	Suspensión de Supabase	El plan gratuito pausa proyectos tras una semana sin actividad. El ping programado del RNF-02 golpea también la base de datos para evitarlo.
RNF-08	Disponibilidad	No hay compromiso de disponibilidad. Es una herramienta interna; una caída temporal es aceptable.
RNF-09	Accesibilidad	Contraste mínimo AA, navegación por teclado en formularios y tablero, etiquetas ARIA en los controles del Kanban.
RNF-10	Idioma	Interfaz y documentación en español. Código, nombres de tablas, campos y endpoints en inglés.
RNF-11	Trazabilidad	Cada entidad guarda <code>created_at</code> , <code>updated_at</code> y el usuario creador. No se implementa una bitácora de auditoría completa.

5. Reglas de negocio

Esta sección es la capa normativa del sistema y prevalece sobre cualquier otra. Las reglas se verifican **siempre en el backend**; ocultar un botón en el frontend no es una medida de seguridad, es una mejora de la experiencia.

5.1 Catálogo de permisos de proyecto

Lista **cerrada** de catorce códigos. Los roles personalizados se arman escogiendo de aquí; no se pueden inventar permisos nuevos sin cambiar el código y esta especificación.

Código	Categoría	Qué habilita
task.view	task	Ver el proyecto, su tablero y sus tareas. Sin este permiso el usuario no ve nada del proyecto.
task.comment	task	Comentar en las tareas.
task.create	task	Crear tareas.
task.edit_any	task	Editar título, descripción, periodicidad y fecha límite de cualquier tarea.
task.delete	task	Eliminar tareas (borrado lógico) y moderar comentarios ajenos.
task.assign	task	Agregar o quitar responsables de una tarea.
task.change_status_any	task	Mover cualquier tarea entre estados, incluidas las ajenas.
task.review	task	Aprobar o devolver entregas.
member.add	member	Agregar al proyecto usuarios que ya existen en la plataforma.
member.remove	member	Retirar miembros del proyecto.
role.assign	role	Cambiar el rol de proyecto de un miembro.
role.manage	role	Crear, editar y eliminar roles personalizados del proyecto.
project.edit	project	Editar nombre, descripción y fechas del proyecto.
project.archive	project	Archivar y desarchivar el proyecto.

RN-05. task.view es obligatorio en todo rol: un rol sin él se rechaza al crearlo. **RN-06.** Los permisos no se implican entre sí en el código; fuera de RN-05, cada permiso se verifica de forma independiente.

5.2 Roles predeterminados de proyecto

Se crean automáticamente en la misma transacción que crea el proyecto, con `is_system = true`: no se pueden eliminar, renombrar ni alterar su conjunto de permisos (RN-23).

Rol	Color	Permisos
Líder	#1F6F5C	Los catorce del catálogo.
Colaborador	#2E5C8A	task.view, task.comment
Observador	#6B6B6B	task.view, task.comment

Colaborador y Observador comparten el mismo conjunto de permisos *a propósito*. La diferencia no está en el catálogo sino en las reglas implícitas de propiedad (5.4): a un Colaborador se le asignan tareas y por eso puede mover y entregar las suyas; un Observador normalmente no tiene tareas asignadas. Si a un Observador se le asigna una, adquiere sobre ella las mismas facultades implícitas. Se distinguen para que el tablero y los filtros comuniquen la intención del líder.

5.3 Ciclo de vida de la tarea

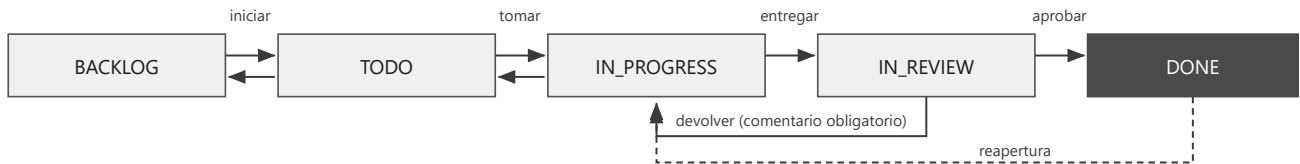


Figura 1. Máquina de estados de la tarea. Las flechas hacia la izquierda en la fila superior son los retrocesos permitidos; la línea discontinua es la reapertura desde DONE.

Transiciones permitidas

Desde	Hacia	Quién	Condición
BACKLOG	TODO	task.change_status_any	—
TODO	IN_PROGRESS	Responsable, o task.change_status_any	—
TODO	BACKLOG	task.change_status_any	—
IN_PROGRESS	TODO	Responsable, o task.change_status_any	—
IN_PROGRESS	IN_REVIEW	Responsable, o task.change_status_any	Exige registrar una entrega con descripción no vacía.
IN_REVIEW	DONE	task.review	Marca la entrega como aprobada y fija completed_at.
IN_REVIEW	IN_PROGRESS	task.review	Devolución: exige comentario de revisión no vacío.
DONE	IN_PROGRESS	task.change_status_any	Reapertura. completed_at vuelve a nulo.

Cualquier transición que no aparezca en esta tabla se rechaza con 409 `INVALID_TRANSITION`, y el mensaje enumera las transiciones válidas desde el estado actual. En particular no se salta de TODO a DONE, ni de IN_REVIEW a TODO, ni de BACKLOG a IN_PROGRESS.

RN-07. Solo un usuario con `task.review` puede sacar una tarea de IN_REVIEW, y quien entregó no puede aprobarse a sí mismo ni aunque tenga el permiso: si el revisor es también responsable de la tarea, la aprobación se rechaza con 403 `CANNOT_REVIEW_OWN_SUBMISSION`. Esto obliga a que exista un segundo par de ojos.

RN-08. Al aprobar se registra `completed_at`; al reabrir vuelve a nulo.

RN-09. Todas las entregas se conservan. Una devolución no borra la anterior: queda en estado REJECTED y la nueva se apila encima.

5.4 Reglas implícitas de propiedad

Independientes del catálogo de permisos. Aplican a todo miembro del proyecto con `task.view`.

ID	Regla
RN-10	Un responsable puede mover sus propias tareas entre TODO e IN_PROGRESS, y llevarlas a IN_REVIEW registrando la entrega, sin necesitar <code>task.change_status_any</code> .
RN-11	El autor de un comentario puede editarlo o borrarlo. Nadie más puede editar comentarios ajenos; quien tenga <code>task.delete</code> puede borrarlos por moderación.
RN-12	Un usuario puede editar su propia entrega mientras la tarea siga en IN_REVIEW y nadie la haya revisado.

5.5 Reglas de proyectos y de roles

ID	Regla
RN-01	El rol global ADMIN no otorga permisos de escritura dentro de un proyecto; sí lectura universal para poder supervisar.
RN-02	Siempre debe existir al menos un usuario ADMIN activo. Toda operación que dejaría el sistema sin administradores se rechaza (409 LAST_ADMIN).
RN-03	Un rol de proyecto pertenece a un único proyecto. No hay roles compartidos ni plantillas globales, y un usuario tiene un solo rol por proyecto.
RN-04	Ser Líder del proyecto A no otorga absolutamente nada en el proyecto B.
RN-13	Solo ADMIN crea proyectos, y al crearlos designa un líder inicial que queda como miembro con el rol Líder.
RN-14	Un proyecto no puede quedarse sin ningún miembro con <code>project.archive</code> . Retirar al último líder o cambiarle el rol se rechaza (409 LAST_PROJECT_ADMIN). El ADMIN siempre puede reasignar el liderazgo.
RN-15	Un proyecto archivado es de solo lectura absoluta: no admite crear ni editar tareas, comentar, entregar, revisar, agregar miembros ni modificar roles. Solo se permite desarchivarlo.
RN-16	Archivar un proyecto detiene toda notificación relacionada, incluidos los recordatorios de tareas vencidas.
RN-17	Los proyectos y sus tareas nunca se eliminan físicamente.
RN-18	Crear, editar o eliminar roles personalizados requiere <code>role.manage</code> , que por defecto solo tiene el Líder.
RN-19	El sistema guarda <code>created_by</code> y <code>created_at</code> de cada rol personalizado, y la interfaz muestra quién lo creó.
RN-20	El nombre de un rol es único dentro del proyecto, sin distinguir mayúsculas.
RN-21	Un rol con miembros asignados no puede eliminarse; hay que reasignarlos primero.
RN-22	Editar los permisos de un rol surte efecto inmediato para todos sus miembros, sin volver a iniciar sesión. Por eso los permisos no se guardan dentro del JWT: se consultan en cada petición.
RN-23	Los roles del sistema (<code>is_system = true</code>) no admiten edición ni eliminación.

5.6 Reglas de notificación e idempotencia

ID	Regla
RN-24	Toda notificación se entrega por dos canales: registro dentro de la aplicación y correo electrónico. El registro se crea siempre; el correo puede fallar sin afectar nada más.
RN-26	La configuración de recordatorios es única para toda la plataforma y solo la modifica un ADMIN: <code>reminder_days_before</code> (por defecto [3,1,0]), <code>send_hour</code> (0-23, por defecto 7), <code>timezone</code> fijo en America/Bogota y <code>overdue_enabled</code> (por defecto verdadero).

RN-27	Idempotencia. Un mismo usuario no recibe dos veces el mismo tipo de recordatorio para la misma tarea el mismo día. Se garantiza con la restricción de unicidad (<code>user_id</code> , <code>task_id</code> , <code>kind</code> , <code>target_date</code>) de la tabla de despachos, no con una comprobación en Python.
RN-28	No se notifica sobre tareas en DONE, ni de proyectos archivados, ni a usuarios desactivados.
RN-29	Si el proceso programado se salta un día, al reanudarse solo envía lo que corresponde al día en curso. No reconstruye avisos atrasados, para no inundar de correos.
RN-30	Un fallo de envío de correo se registra y se reintenta hasta 3 veces con espera creciente. Nunca revierte ni bloquea la operación de negocio que lo originó.

RN-25 — Eventos que generan notificación:

Evento	Destinatarios
Usuario agregado como responsable de una tarea	El responsable agregado
Recordatorio previo al vencimiento	Todos los responsables activos de la tarea
Tarea vencida	Todos los responsables activos más los miembros con <code>task.review</code> del proyecto
Entrega aprobada o devuelta	Quien registró la entrega
Invitación a la plataforma	El invitado (solo correo; todavía no existe la cuenta)
Restablecimiento de contraseña	El solicitante (solo correo)

5.7 Reglas de autenticación

ID	Regla
RN-37	No existe el registro abierto. La única vía de entrada es una invitación creada por un ADMIN.
RN-38	El token de invitación vive 7 días, es de un solo uso y se almacena hashado. Reenviar una invitación invalida el token anterior.
RN-39	Aceptar una invitación exige nombre completo y contraseña de mínimo 10 caracteres. La cuenta se crea en ese momento; antes de aceptar, el usuario no existe.
RN-40	No hay verificación de correo aparte: recibir y usar el enlace de invitación ya prueba control de la dirección.
RN-41	El token de restablecimiento vive 1 hora, es de un solo uso y se guarda hashado. Solicitar el restablecimiento devuelve siempre la misma respuesta, exista o no la cuenta.
RN-42	Restablecer o cambiar la contraseña revoca todos los tokens de refresco vigentes del usuario.
RN-43	El token de refresco es rotativo: cada uso emite uno nuevo y revoca el anterior. Si se detecta el uso de un token ya revocado, se revocan todas las sesiones de ese usuario.
RN-44	Un usuario desactivado no puede iniciar sesión y sus tokens de refresco se revocan en el acto.

5.8 Matriz de autorización resumida

Operación	Requisito
Invitar usuario a la plataforma	Rol global ADMIN
Desactivar usuario · cambiar rol global	Rol global ADMIN
Crear proyecto	Rol global ADMIN
Ver un proyecto	Miembro con <code>task.view</code> , o rol global ADMIN (solo lectura)

Editar proyecto	project.edit
Archivar · desarchivar	project.archive
Agregar miembro al proyecto	member.add
Retirar miembro	member.remove
Cambiar rol de un miembro	role.assign
Crear · editar · borrar rol personalizado	role.manage
Crear tarea	task.create
Editar cualquier tarea	task.edit_any
Eliminar tarea	task.delete
Asignar responsables	task.assign
Mover tarea propia (TODO ↔ IN_PROGRESS → IN_REVIEW)	Ser responsable
Mover cualquier tarea	task.change_status_any
Aprobar o devolver entrega	task.review y no ser responsable de esa tarea
Comentar	task.comment
Configurar recordatorios	Rol global ADMIN
Crear · editar · publicar lecciones	Rol global ADMIN o LESSON_EDITOR
Ver lecciones publicadas	Cualquier usuario autenticado

6. Arquitectura del sistema

6.1 Vista general

Monolito modular en el backend, aplicación de página única en el frontend, base de datos gestionada. Tres piezas desplegadas por separado más un proceso programado externo. La arquitectura se documenta con el modelo **C4**: contexto (6.2), contenedores (6.3), componentes (6.4) y estructura de código (6.5).

6.2 C4 nivel 1 — Contexto

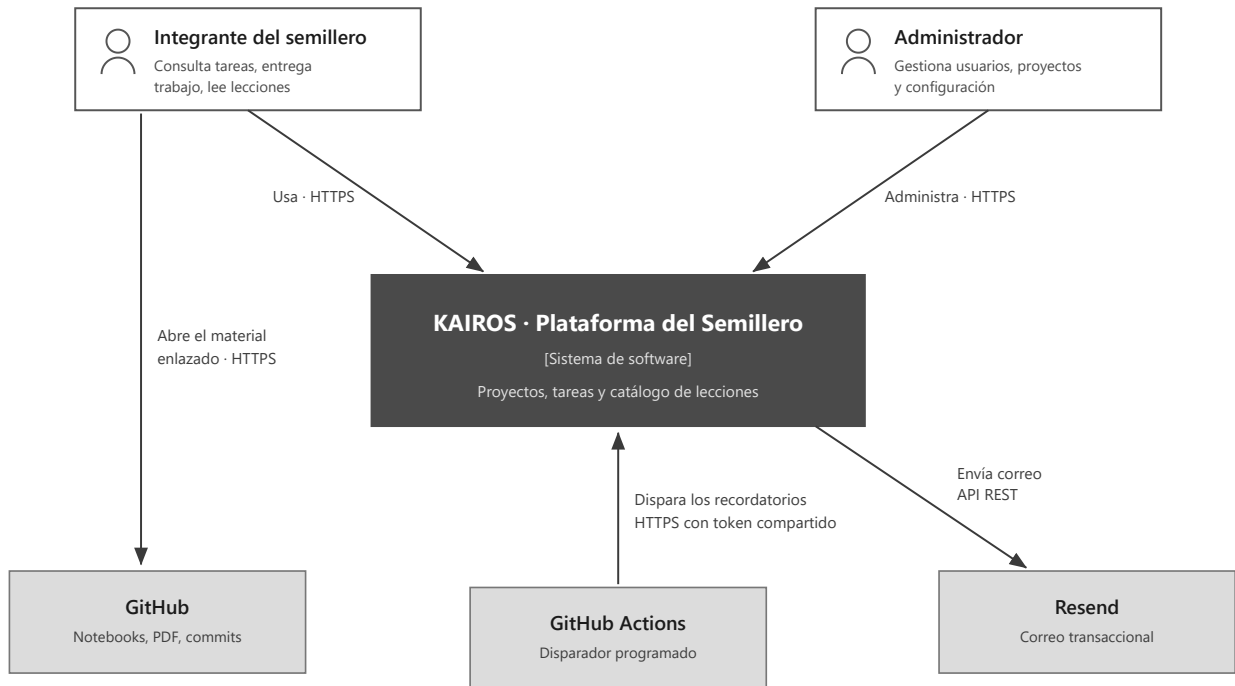


Figura 2. C4 nivel 1 — Contexto. La plataforma no aloja archivos: el material vive en GitHub y el usuario lo abre directamente.

6.3 C4 nivel 2 — Contenedores

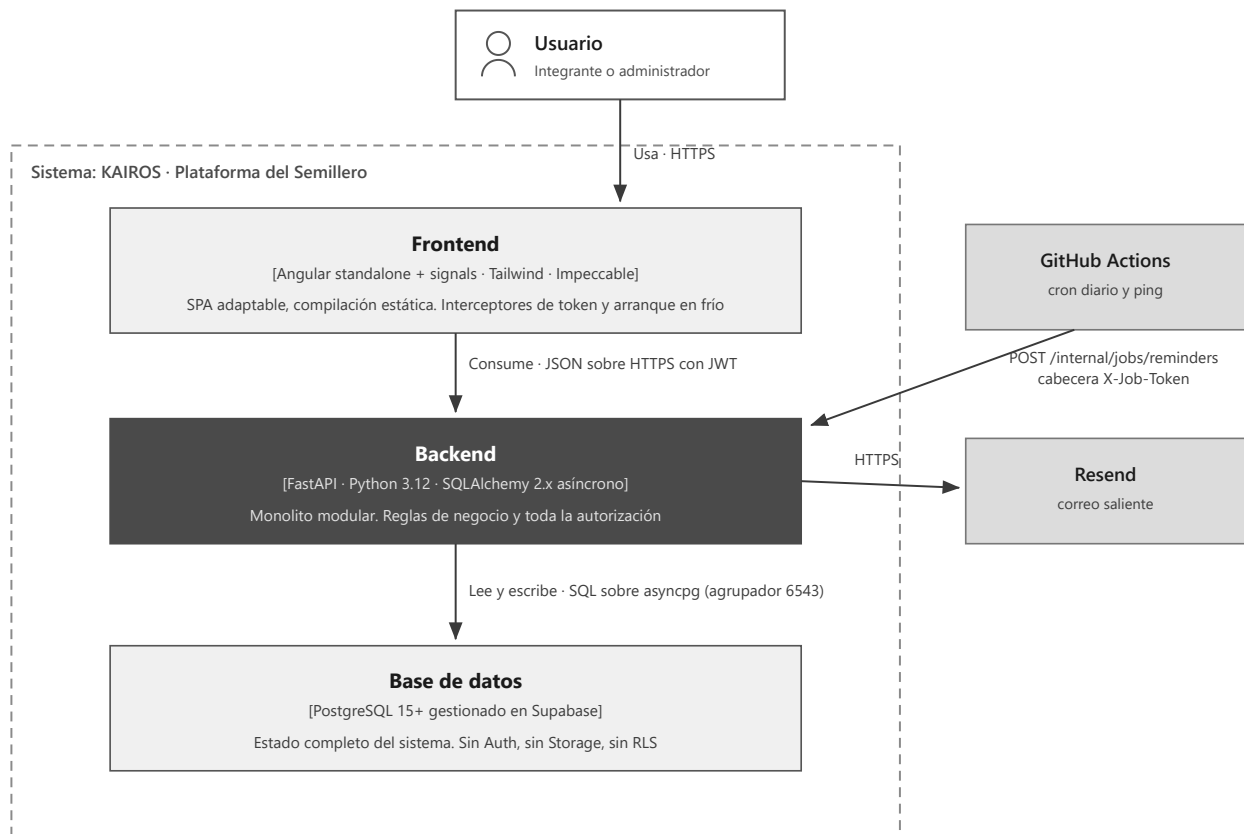


Figura 3. C4 nivel 2 — Contenedores. El frontend nunca habla con la base de datos: toda regla y toda autorización viven en el backend.

6.4 C4 nivel 3 — Componentes del backend

Este nivel es el que define **la estructura del proyecto** del lado del servidor. Cada módulo de `modules/` es una unidad con las mismas seis piezas y fronteras estrictas entre ellas (6.6).



Figura 4. C4 nivel 3 — Componentes del backend. Solo los `repository.py` tocan la base de datos; los `router.py` no consultan y los `service.py` no conocen HTTP.

6.5 Estructura del proyecto (C4 nivel 4 — código)

KAIROS/	
├─ CLAUDE.md	Convenciones obligatorias para quien escriba código
├─ README.md	
├─ docs/	PRD, reglas, modelo de datos, arquitectura, API, historias
├─ .github/workflows/	
│ └─ reminders.yml	cron diario 12:00 UTC = 07:00 America/Bogota
│ └─ keepalive.yml	ping cada 10 min en horario hábil
├─ backend/	
│ └─ app/	
│ └─ main.py	aplicación, middlewares, routers, manejadores de excepción
│ └─ core/	
│ └─ config.py	ajustes con pydantic-settings
│ └─ database.py	motor asíncrono, sesión, get_db
│ └─ security.py	hash Argon2, emisión y validación de JWT
│ └─ dependencies.py	get_current_user, require_global_role, require_project_permission
│ └─ exceptions.py	excepciones de dominio y sus manejadores HTTP
│ └─ pagination.py	
│ └─ modules/	
│ └─ auth/	router · service · repository · models · schemas · exceptions
│ └─ users/	
│ └─ projects/	incluye roles, permisos y membresías
│ └─ tasks/	incluye entregas y comentarios
│ └─ lessons/	
│ └─ notifications/	
│ └─ jobs/	
│ └─ reminders.py	lógica del trabajo programado
│ └─ integrations/	
│ └─ email/	
│ └─ client.py	cliente de Resend
│ └─ templates/	plantillas HTML de correo
│ └─ cli.py	create_admin y utilidades de mantenimiento
├─ alembic/	una migración por cambio, con mensaje descriptivo
├─ tests/	
│ └─ unit/	servicios con repositorios simulados
│ └─ integration/	endpoints contra PostgreSQL real
├─ pyproject.toml	
├─ .env.example	
├─ frontend/	
│ └─ src/app/	
│ └─ core/	
│ └─ auth/	servicio de sesión, guardas, interceptor
│ └─ api/	clientes HTTP tipados (ningún componente usa HttpClient)
│ └─ layout/	estructura general, barra lateral, campana
│ └─ features/	
│ └─ auth/	ingreso, aceptar invitación, recuperar contraseña
│ └─ dashboard/	«Mis tareas» entre todos los proyectos
│ └─ projects/	lista, detalle, tablero, miembros, roles
│ └─ lessons/	catálogo y edición
│ └─ admin/	usuarios, invitaciones, configuración
│ └─ shared/	
│ └─ ui/	componentes reutilizables
│ └─ pipes/	
├─ PRODUCT.md	generado por Impeccable
├─ DESIGN.md	tokens de diseño (no se inventan a mano)
└─ package.json	

Figura 5. Estructura del repositorio. Los nombres de archivo van en *kebab-case* en el frontend y en *snake_case* en el backend.

6.6 Fronteras entre módulos

Estas seis reglas son lo que hace que el monolito sea *modular* en lugar de un montón de archivos. El objetivo no es prepararse para microservicios, sino poder razonar sobre cada módulo por separado.

1. **Un router nunca consulta la base de datos.** Llama al servicio y traduce el resultado a HTTP.
2. **Un servicio nunca conoce HTTP.** No recibe Request ni devuelve JSONResponse: levanta excepciones de dominio que el manejador global convierte en códigos de estado.
3. **Un repositorio nunca decide.** Recibe parámetros, ejecuta la consulta, devuelve datos. Ninguna condición de negocio vive ahí.
4. **Un módulo no importa el repositorio ni los modelos de otro módulo.** Si tasks necesita algo de projects, llama a `projects.service`. La única excepción son las llaves foráneas por identificador.
5. **No hay relaciones de SQLAlchemy que crucen módulos.** Task no declara `relationship("Project")`: guarda `project_id` y punto. Así una consulta no arrastra media base de datos sin que nadie se dé cuenta.
6. **Las transacciones se abren y se confirman en el servicio,** nunca en el repositorio ni en el router.

Dependencias permitidas entre módulos:

auth	→	users
users	→	(ninguno)
projects	→	users
tasks	→	projects, notifications
lessons	→	(ninguno)
notifications	→	(ninguno; recibe los datos, no los busca)

notifications no importa nada de nadie: los demás módulos le pasan la información que debe enviar. Así se puede probar en aislamiento y no se convierte en el nudo donde todo se enreda.

6.7 Autenticación y autorización

Flujo de tokens

- **Token de acceso:** JWT firmado con HS256, vigencia 15 minutos. Contenido: `sub` (id de usuario), `global_role`, `exp`, `iat`, `jti`.
- **Token de refresco:** cadena aleatoria de 32 bytes, vigencia 7 días, guardada hasheada con SHA-256. Rotativa: cada uso emite una nueva y revoca la anterior (RN-43).
- **Almacenamiento en el cliente:** ambos en memoria dentro de un servicio de Angular; el de refresco se persiste en `localStorage` para sobrevivir a la recarga. Es una herramienta interna sin datos sensibles y la complejidad de cookies `HttpOnly` con CSRF no se justifica. Queda registrado como decisión consciente, no como descuido.

Por qué los permisos no van dentro del token

El JWT lleva **solo** el rol global. Los permisos de proyecto se consultan en cada petición porque un líder puede cambiarlos en cualquier momento y el cambio debe surtir efecto de inmediato (RN-22). Meterlos en el token obligaría a esperar quince minutos o a implementar una lista de revocación, lo cual es más complejo que una consulta indexada.

Dependencia única de autorización

Toda verificación de permisos de proyecto pasa por aquí. **Nunca** se comprueban permisos con condicionales sueltos dentro de un servicio.

```
def require_project_permission(permission: str):
    async def dependency(
        project_id: UUID,
        user: User = Depends(get_current_user),
        db: AsyncSession = Depends(get_db),
    ) -> ProjectContext:
        ctx = await projects_service.get_context(db, project_id, user.id)
        if ctx is None:
            raise NotFoundError("Proyecto no encontrado") # 404, no 403
        if permission not in ctx.permissions:
            raise ForbiddenError(f"Se requiere el permiso {permission}")
        if ctx.project.status == ProjectStatus.ARCHIVED and permission != "project.archive":
            raise ConflictError("El proyecto está archivado")
        return ctx
    return dependency
```

Tres detalles que importan: se devuelve **404 en lugar de 403** cuando el usuario no es miembro, porque un 403 confirmaría que el proyecto existe; la **comprobación de archivado está centralizada** aquí y no repartida por cada endpoint, lo que cumple RN-15 sin depender de que nadie se acuerde; y ProjectContext trae el proyecto, la membresía y el conjunto de permisos ya resueltos, para que el servicio no vuelva a consultarlos.

6.8 Proceso programado de recordatorios

La restricción de fondo es que **Render no ofrece trabajos programados en el plan gratuito**. La solución es un disparador externo contra un endpoint interno autenticado con un secreto comparado en tiempo constante y excluido del esquema público de OpenAPI.

```
POST /internal/jobs/reminders
Header: X-Job-Token: <secreto compartido>
```

El flujo de GitHub Actions despierta primero la API con un curl a /health y solo después dispara el trabajo; si el disparo llegara directo, la primera petición se agotaría antes de que el servicio arrancara. Un segundo flujo golpea /health cada diez minutos entre las 6:00 y las 23:00 hora de Colombia, lo que evita a la vez la suspensión de Render (RNF-02) y la pausa por inactividad de Supabase (RNF-07).

Algoritmo

```
config ← notification_settings
hoy ← fecha actual en America/Bogota

para cada d en config.reminder_days_before:
    tareas ← SELECT de tasks
              WHERE due_date = hoy + d
              AND status <> 'DONE'
              AND deleted_at IS NULL
              AND project.status = 'ACTIVE'
    para cada tarea, para cada responsable activo:
        despachar(usuario, tarea, 'TASK_DUE_SOON', hoy)

si config.overdue_enabled:
    tareas ← ... WHERE due_date < hoy AND (mismas condiciones)
    destinatarios ← responsables activos U miembros con task.review
    para cada destinatario:
        despachar(destinatario, tarea, 'TASK_OVERDUE', hoy)

función despachar(usuario, tarea, tipo, fecha):
    INSERT INTO notification_dispatches (...) ON CONFLICT DO NOTHING RETURNING id
    si no devolvió fila: return # ya se envió hoy, se omite
    crear la notificación dentro de la aplicación
    enviar el correo; actualizar status y provider_message_id
```

La inserción va antes del envío. El `ON CONFLICT DO NOTHING` sobre la restricción `UNIQUE (user_id, task_id, kind, target_date)` es lo que hace el trabajo idempotente (RN-27). Si el proceso se ejecuta dos veces, la segunda no inserta y por lo tanto no envía. Delegar esto a la base de datos es más confiable que cualquier comprobación previa en Python, que siempre deja una ventana de carrera.

Advertencia sobre el cron de GitHub Actions: las ejecuciones programadas suelen retrasarse varios minutos y, con mucha carga, pueden saltarse. Para recordatorios diarios es tolerable; es exactamente la razón por la que existe RN-29.

6.9 Frontend

- **Componentes standalone y signals.** Sin `NgModules` y sin `NgRx`: `signal`, `computed` y `resource` bastan para esta escala.
- **Rutas con carga diferida** por funcionalidad, para que el paquete inicial sea pequeño.
- **Interceptor HTTP** que adjunta el token de acceso, renueva de forma transparente ante un 401 y encola las peticiones concurrentes durante la renovación para no disparar varios refrescos a la vez.
- **Interceptor de arranque en frío:** si una petición pasa de 3 segundos emite un evento global que muestra «Despertando el servidor...»; el tiempo de espera es de 90 segundos y nunca se muestra un error genérico ni una pantalla en blanco.
- **Impeccable manda en todo lo visual.** Los tokens de diseño los define `DESIGN.md`; no se inventan a mano. Anti-patrones prohibidos: tipografías por defecto del sistema, degradados de morado a azul, tarjetas anidadas, texto gris sobre fondos de color, negros y grises puros, y suavizados con rebote.

Estrategia adaptable (RNF-01)

Ancho	Tablero	Navegación
< 768 px	Lista agrupada por estado, con selector desplegable para cambiar de estado. Sin arrastrar y soltar.	Barra inferior

768 – 1024 px	Columnas con desplazamiento horizontal, arrastrar y soltar activo.	Barra lateral colapsable
> 1024 px	Las cinco columnas visibles, arrastrar y soltar.	Barra lateral fija

El arrastrar y soltar se desactiva por debajo de 768 px: en móvil es incómodo y propenso a errores, y el selector de estado resulta más rápido y más accesible. Las áreas táctiles miden al menos 44 × 44 px.

6.10 Manejo de errores

Formato uniforme para toda respuesta de error. Los mensajes van en español porque se muestran al usuario; los códigos van en inglés porque los consume el frontend.

```
{
  "error": {
    "code": "PROJECT_ARCHIVED",
    "message": "El proyecto está archivado y no admite modificaciones",
    "details": null
  }
}
```

Excepción de dominio	HTTP	Código
NotFoundError	404	NOT_FOUND
ForbiddenError	403	FORBIDDEN
ConflictError	409	CONFLICT y sus especializaciones
ValidationError	422	VALIDATION_ERROR
AuthenticationError	401	UNAUTHENTICATED

7. Modelo de datos

PostgreSQL 15+ en Supabase, en tercera forma normal, con claves primarias uuid generadas con `gen_random_uuid()` y todas las marcas de tiempo en `timestampz` UTC.

7.1 Convenciones

- Tablas en plural y `snake_case`, en inglés.
- Toda tabla de negocio lleva `created_at timestampz NOT NULL DEFAULT now()`; las mutables llevan además `updated_at`, mantenido por un trigger `set_updated_at()`.
- El borrado lógico usa `deleted_at timestampz NULL` y **solo lo tienen tasks y task_comments**.
- Los enumerados se implementan como tipos `ENUM` cuando el conjunto es cerrado y estable, y como tabla catálogo cuando debe consultarse o referenciarse (caso de `permissions`).
- Las llaves foráneas usan `ON DELETE RESTRICT` por defecto; `CASCADE` solo donde el hijo carece de sentido sin el padre, y siempre señalado explícitamente.

7.2 Tipos enumerados

```
CREATE TYPE global_role          AS ENUM ('ADMIN', 'LESSON_EDITOR', 'MEMBER');
CREATE TYPE user_status          AS ENUM ('ACTIVE', 'DISABLED');
CREATE TYPE invitation_status    AS ENUM ('PENDING', 'ACCEPTED', 'REVOKED', 'EXPIRED');
CREATE TYPE project_status      AS ENUM ('ACTIVE', 'ARCHIVED');
CREATE TYPE task_status          AS ENUM ('BACKLOG', 'TODO', 'IN_PROGRESS', 'IN_REVIEW', 'DONE');
CREATE TYPE task_periodicity     AS ENUM ('ONE_TIME', 'WEEKLY', 'MONTHLY', 'SEMESTER');
CREATE TYPE submission_review_status AS ENUM ('PENDING', 'APPROVED', 'REJECTED');
CREATE TYPE resource_type        AS ENUM ('NOTEBOOK', 'PDF', 'VIDEO', 'REPOSITORY', 'ARTICLE');
CREATE TYPE notification_kind    AS ENUM ('TASK_ASSIGNED', 'TASK_DUE_SOON', 'TASK_OVERDUE',
                                          'SUBMISSION_APPROVED', 'SUBMISSION_REJECTED');
CREATE TYPE dispatch_status      AS ENUM ('SENT', 'FAILED');
```

7.3 Tablas

Identidad y acceso

Tabla	Columnas y restricciones
users	<code>id</code> uuid PK · <code>full_name</code> text (2-120) · <code>email</code> citext UNIQUE · <code>password_hash</code> text · <code>global_role</code> (def. MEMBER) · <code>status</code> (def. ACTIVE) · <code>last_login_at</code> · <code>created_at/updated_at</code> . <i>Índices:</i> UNIQUE (email), INDEX (status) WHERE status = 'ACTIVE'. Se usa citext para que el correo sea único sin distinguir mayúsculas; alternativa si no está disponible: índice único sobre lower(email).
invitations	<code>id</code> · <code>email</code> citext · <code>global_role</code> · <code>token_hash</code> text UNIQUE · <code>expires_at</code> · <code>status</code> (def. PENDING) · <code>invited_by</code> FK→users · <code>accepted_at</code> · <code>accepted_user_id</code> FK→users · <code>created_at</code> . <i>Restricción:</i> UNIQUE (email) WHERE status = 'PENDING' (índice único parcial): una sola invitación viva por correo. <i>Decisión:</i> el usuario no se crea al invitar sino al aceptar, para no contaminar los selectores de asignación con filas a medias.
refresh_tokens	<code>id</code> · <code>user_id</code> FK→users ON DELETE CASCADE · <code>token_hash</code> UNIQUE · <code>expires_at</code> · <code>revoked_at</code> · <code>user_agent</code> · <code>created_at</code> . <i>Índice:</i> (user_id) WHERE revoked_at IS NULL.

password_reset_tokens	Misma forma que refresh_tokens, con used_at en lugar de revoked_at y vigencia de 1 hora.
-----------------------	--

Proyectos y roles

Tabla	Columnas y restricciones
projects	id · name (3-120) · description · status (def. ACTIVE) · start_date date · created_by FK→users · archived_at · created_at/updated_at. <i>Restricción:</i> CHECK ((status = 'ARCHIVED') = (archived_at IS NOT NULL)), que impide que el estado y la fecha se desincronicen. <i>Índice:</i> (status).
permissions	code text PK (ej. task.review) · description · category (task, member, role, project). Tabla catálogo poblada por migración con los 14 códigos de la sección 5.1. No se modifica en tiempo de ejecución.
project_roles	id · project_id FK→projects CASCADE · name (2-40) · color CHECK (color ~ '^#[0-9A-Fa-f]{6}\$') · is_system bool · created_by FK→users NULL (nulo en los roles del sistema) · created_at/updated_at. <i>Restricción:</i> UNIQUE (project_id, lower(name)) (RN-20).
project_role_permissions	project_role_id FK→project_roles CASCADE · permission_code FK→permissions. PK compuesta. Es la tabla puente que hace configurable el sistema de permisos.
project_members	id · project_id FK CASCADE · user_id FK→users · project_role_id FK→project_roles · added_by FK→users · joined_at. <i>Restricción:</i> UNIQUE (project_id, user_id) — un usuario tiene un único rol por proyecto (RN-03). <i>Índices:</i> (user_id) para «Mis proyectos», (project_id). <i>Integridad que la base no expresa sola:</i> project_role_id debe pertenecer al mismo project_id. Se valida en el servicio y se refuerza opcionalmente con una FK compuesta añadiendo UNIQUE (id, project_id) en project_roles.

Tareas

Tabla	Columnas y restricciones
tasks	id · project_id FK→projects · title (3-200) · description · status (def. BACKLOG) · periodicity (def. ONE_TIME) · due_date date · created_by FK→users · completed_at · deleted_at · created_at/updated_at. <i>Restricción:</i> CHECK ((status = 'DONE') = (completed_at IS NOT NULL)) (RN-08). <i>Índices:</i> (project_id, status) WHERE deleted_at IS NULL — la consulta del tablero; (due_date) WHERE deleted_at IS NULL AND status <> 'DONE' — la consulta del proceso programado. <i>Por qué due_date es date y no timestamp tz:</i> el vencimiento es un día calendario en hora de Colombia, no un instante. Evita el error clásico de que una tarea «venza» a las 7 p. m. del día anterior por conversión de zona horaria.
task_assignees	task_id FK→tasks CASCADE · user_id FK→users · assigned_by FK→users · assigned_at. PK compuesta (task_id, user_id); índice adicional en (user_id) para la vista «Mis tareas».
task_submissions	id · task_id FK CASCADE · submitted_by FK→users · description (10-4000) · commit_url CHECK (commit_url IS NULL OR commit_url ~ '^https://github\.com/') · review_status (def. PENDING) · reviewed_by · reviewed_at · review_comment · created_at. <i>Restricciones:</i> CHECK ((review_status = 'PENDING') = (reviewed_at IS NULL)) y CHECK (review_status <> 'REJECTED' OR review_comment IS NOT NULL) — devolver exige explicación. <i>Índice:</i> UNIQUE (task_id) WHERE review_status = 'PENDING': impide dos entregas pendientes sobre la misma tarea.

task_comments	id · task_id FK CASCADE · author_id FK→users · body (1-4000) · deleted_at · created_at/updated_at. <i>Índice:</i> (task_id, created_at) WHERE deleted_at IS NULL.
---------------	---

Notificaciones

Tabla	Columnas y restricciones
notification_settings	id smallint PK CHECK (id = 1) · reminder_days_before int[] (def. {3,1,0}) · send_hour smallint (def. 7, CHECK 0-23) · timezone (def. America/Bogota) · overdue_enabled bool (def. true) · updated_by · updated_at. El truco de CHECK (id = 1) fuerza que exista una sola configuración global, tal como pide RF-43.
notifications	id · user_id FK CASCADE · kind · title · body · task_id FK NULL CASCADE · project_id FK NULL CASCADE · read_at · created_at. <i>Índices:</i> (user_id, created_at DESC) y (user_id) WHERE read_at IS NULL para el contador de no leídas.
notification_dispatches	id · user_id FK · task_id FK NULL · kind · target_date date · status · provider_message_id · error_message · attempts smallint (def. 1) · created_at. Restricción crítica: UNIQUE (user_id, task_id, kind, target_date). El proceso programado intenta insertar aquí antes de enviar; si la inserción viola la restricción, el aviso ya salió y se omite. Es idempotencia garantizada por la base de datos y no por lógica de aplicación, que es lo que la hace confiable ante reintentos y ejecuciones concurrentes.

Lecciones

Tabla	Columnas y restricciones
lesson_modules	id · title (3-150) · description · position int · is_published bool (def. false) · created_by FK→users · created_at/updated_at.
lessons	id · module_id FK→lesson_modules CASCADE · title · description · position · is_published · created_by · created_at/updated_at. <i>Índice:</i> (module_id, position).
lesson_resources	id · lesson_id FK→lessons CASCADE · type · title · url CHECK (url ~ '^https?://') · position · created_at.

7.4 Mapa entidad-relación

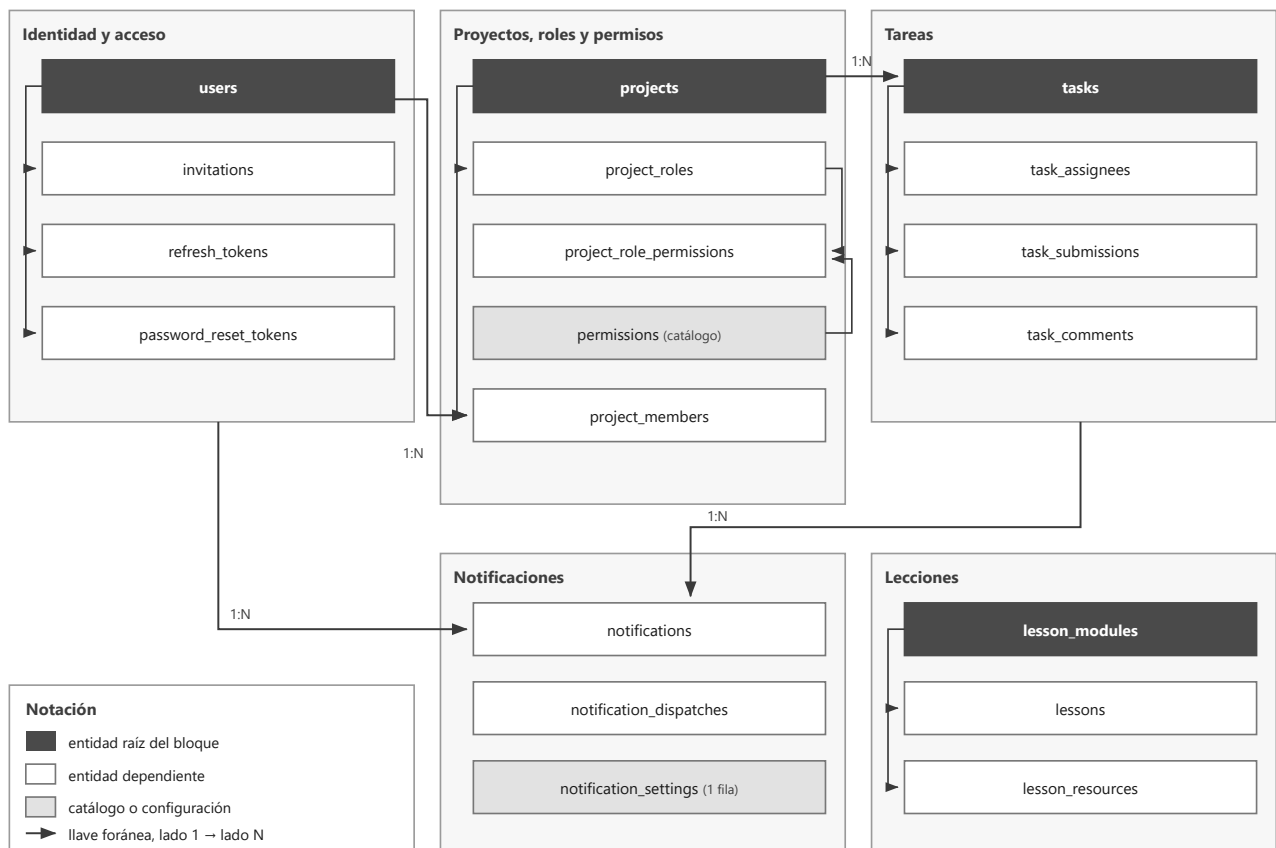


Figura 6. Mapa de tablas por módulo. Las relaciones dibujadas son las estructurales; las referencias de auditoría a `users` (`created_by`, `added_by`, `assigned_by`, `reviewed_by`, `submitted_by`, `author_id`) existen en casi todas las tablas y se omiten para no saturar el diagrama.

Relaciones principales

Relación	Cardinalidad	Nota
<code>users</code> → <code>invitations</code> / <code>refresh_tokens</code> / <code>password_reset_tokens</code>	1 : N	Los tokens caen en cascada al borrar el usuario, pero los usuarios nunca se borran (RF-09).
<code>projects</code> → <code>project_roles</code>	1 : N	Cascada. Un rol pertenece a un único proyecto (RN-03).
<code>project_roles</code> ↔ <code>permissions</code>	N : M	Resuelta por <code>project_role_permissions</code> . Es lo que hace configurable el sistema.
<code>projects</code> → <code>project_members</code> ← <code>users</code>	1 : N : 1	UNIQUE (<code>project_id</code> , <code>user_id</code>): un rol por usuario y proyecto.
<code>projects</code> → <code>tasks</code>	1 : N	Sin <code>relationship()</code> de SQLAlchemy: solo <code>project_id</code> .
<code>tasks</code> ↔ <code>users</code>	N : M	Resuelta por <code>task_assignees</code> ; una tarea admite varios responsables (RF-27).
<code>tasks</code> → <code>task_submissions</code> / <code>task_comments</code>	1 : N	Historial completo, nunca solo la última entrega (RF-34, RN-09).
<code>tasks</code> → <code>notifications</code> / <code>notification_dispatches</code>	1 : N	<code>task_id</code> nulo para notificaciones no ligadas a una tarea.

lesson_modules → lessons → lesson_resources	1 : N : N	Cascada en ambos niveles (RN-36); la interfaz advierte el conteo exacto antes de borrar.
--	-----------	--

7.5 Notas de normalización

- **1FN.** Ninguna columna guarda listas, con una excepción deliberada: `notification_settings.reminder_days_before` es un `int[]`. Se justifica porque es una fila única de configuración, el arreglo nunca se consulta ni se une con otras tablas, y una tabla aparte para tres enteros sería ceremonia sin beneficio. Queda documentado como excepción consciente.
- **2FN.** Las tablas puente (`task_assignees`, `project_role_permissions`) contienen únicamente atributos que dependen de la clave compuesta completa: `assigned_by` y `assigned_at` dependen del par tarea-usuario, no de uno solo.
- **3FN.** No se guardan datos derivados. En particular **no** existen `tasks.assignee_count`, `projects.member_count` ni `tasks.is_overdue`. Lo vencido se calcula comparando `due_date` con la fecha actual en America/Bogota; el conteo de miembros se calcula al consultar.
- **Redundancia controlada:** ninguna en esta versión. Con 50 usuarios y 1.000 tareas, ninguna consulta justifica desnormalizar.

7.6 Datos iniciales

1. Los catorce registros de `permissions`.
2. La fila única de `notification_settings` con sus valores por defecto.
3. El primer usuario ADMIN, creado con `python -m app.cli create_admin`, **nunca** por un endpoint expuesto.

Los roles de proyecto predeterminados no son datos iniciales globales: se generan por proyecto, dentro de la misma transacción que lo crea.

8. Contrato de la API

Base: <https://api.kairospartners.uk/api/v1>. Autenticación con Authorization: Bearer <access_token> salvo donde se indique. Listas paginadas con ?page=1&size=20, devolviendo { "items": [...], "total": n, "page": p, "size": s }. Los identificadores son UUID; las fechas de vencimiento son YYYY-MM-DD y el resto de marcas de tiempo son ISO 8601 en UTC. Rutas en plural y minúscula; esquemas Pydantic con sufijo por uso (TaskCreate, TaskUpdate, TaskRead).

8.1 Autenticación

Método	Ruta	Acceso	Descripción
POST	/auth/login	Pública	Ingreso con correo y contraseña. Devuelve tokens y el perfil.
POST	/auth/refresh	Pública	Renueva tokens con rotación (RN-43).
POST	/auth/logout	Requerida	Revoca el token de refresco.
GET / PATCH	/auth/me	Requerida	Perfil del usuario en sesión; PATCH edita el nombre propio.
POST	/auth/change-password	Requerida	Cambia la contraseña indicando la actual.
GET	/auth/invitations/{token}	Pública	Valida el token y devuelve el correo asociado.
POST	/auth/invitations/{token}/accept	Pública	Crea la cuenta. Errores: 404, 409 INVITATION_EXPIRED, 409 INVITATION_ALREADY_USED.
POST	/auth/password-reset/request	Pública	Devuelve siempre 202 con el mismo cuerpo, exista o no la cuenta (RN-41).
POST	/auth/password-reset/confirm	Pública	Define la nueva contraseña y revoca todas las sesiones (RN-42).

POST /auth/login responde 401 UNAUTHENTICATED con credenciales incorrectas o cuenta desactivada, y **el mensaje es idéntico en ambos casos** para no revelar qué correos existen.

8.2 Usuarios e invitaciones

Todo este grupo exige rol global ADMIN, salvo GET /users, que cualquier autenticado puede consultar en versión reducida (id, nombre y correo) para los selectores de asignación.

Método	Ruta	Descripción
GET	/users	Lista con ?search=&status=&global_role=.
GET	/users/{id}	Detalle.
PATCH	/users/{id}/role	Cambia el rol global. 409 LAST_ADMIN si dejaría el sistema sin administradores.
PATCH	/users/{id}/status	Activa o desactiva. Misma protección LAST_ADMIN.
GET	/invitations	Lista con ?status=.
POST	/invitations	Crea y envía. 409 EMAIL_ALREADY_REGISTERED si el correo ya tiene cuenta.
POST	/invitations/{id}/resend	Reenvía, invalidando el token anterior.

DELETE	/invitations/{id}	Revoca una invitación pendiente.
--------	-------------------	----------------------------------

```
// POST /invitations → 201
{
  "id": "...",
  "email": "nuevo@ejemplo.com",
  "global_role": "MEMBER",
  "status": "PENDING",
  "expires_at": "2026-09-12T14:00:00Z",
  "invite_url": "https://app.kairospartners.uk/invitacion/a1b2c3..."
}
```

invite_url se devuelve **una sola vez**, para que el administrador pueda copiarla y enviarla por otro canal. Después no se puede recuperar: solo se guarda el hash del token.

8.3 Proyectos, miembros y roles

Método	Ruta	Requisito
GET	/projects	Autenticado. Devuelve los proyectos del usuario; un ADMIN los ve todos. Filtro ?status=.
POST	/projects	Rol global ADMIN
GET	/projects/{id}	task.view
PATCH	/projects/{id}	project.edit
POST	/projects/{id}/archive · /unarchive	project.archive
GET	/projects/{id}/members	task.view
POST	/projects/{id}/members	member.add
DELETE	/projects/{id}/members/{user_id}	member.remove. 409 LAST_PROJECT_ADMIN si dejaría el proyecto sin nadie con project.archive.
PATCH	/projects/{id}/members/{user_id}/role	role.assign. Misma protección LAST_PROJECT_ADMIN.
GET	/projects/{id}/roles	task.view
POST · PATCH · DELETE	/projects/{id}/roles[/role_id]	role.manage. Errores: 409 ROLE_NAME_TAKEN, 422 si falta task.view, 409 ROLE_HAS_MEMBERS, 409 SYSTEM_ROLE_IMMUTABLE.
GET	/permissions	Autenticado. Catálogo completo para armar la interfaz de roles.

POST /projects ejecuta en una sola transacción: crea el proyecto, genera los tres roles predeterminados con sus permisos y agrega al líder designado.

```
// POST /projects
{ "name": "Detección de anomalías", "description": "...",
  "start_date": "2026-09-10", "leader_user_id": "uuid-de-carlos" }

// GET /projects/{id} → incluye los permisos del usuario, para que el frontend sepa qué mostrar
{ "id": "...", "name": "Detección de anomalías", "status": "ACTIVE", "member_count": 5,
  "task_counts": { "BACKLOG": 3, "TODO": 4, "IN_PROGRESS": 2, "IN_REVIEW": 1, "DONE": 12 },
  "my_permissions": ["task.view", "task.comment", "task.create", "..."] }
```

8.4 Tareas, entregas y comentarios

Método	Ruta	Requisito
GET	/projects/{pid}/tasks	task.view. Filtros ? status=&assignee_id=&periodicity=&overdue=true.
POST	/projects/{pid}/tasks	task.create
GET	/tasks/{id}	task.view del proyecto
PATCH · DELETE	/tasks/{id}	task.edit_any · task.delete (borrado lógico)
POST · DELETE	/tasks/{id}/assignees[/ {user_id}]	task.assign
POST	/tasks/{id}/status	Según la máquina de estados (5.3). 409 INVALID_TRANSITION, 409 SUBMISSION_REQUIRED.
GET	/me/tasks	Autenticado. Tareas propias entre todos los proyectos, con ? due_before=&status= y el nombre del proyecto en cada fila.
GET · POST	/tasks/{id}/submissions	task.view · ser responsable de la tarea
PATCH	/submissions/{id}	Autor, solo mientras esté PENDING (RN-12)
POST	/submissions/{id}/review	task.review y no ser responsable (RN-07)
GET · POST	/tasks/{id}/comments	task.view · task.comment
PATCH · DELETE	/comments/{id}	Autor; además task.delete para moderar comentarios ajenos

```
// POST /tasks/{id}/submissions → mueve la tarea a IN_REVIEW en la misma transacción
{ "description": "Entrené el modelo base con los datos de agosto. Exactitud del 0.87 en validación.",
  "commit_url": "https://github.com/semillero-ml/anomalias/commit/a1b2c3d" }
// Errores: 409 SUBMISSION_ALREADY_PENDING · 422 si commit_url no es de GitHub · 403 NOT_ASSIGNEE

// POST /submissions/{id}/review → aprobar lleva a DONE; devolver lleva a IN_PROGRESS
{ "approved": false, "comment": "Falta la evaluación en el conjunto de prueba." }
// Errores: 422 REVIEW_COMMENT_REQUIRED · 403 CANNOT_REVIEW_OWN_SUBMISSION
```

8.5 Notificaciones y lecciones

Método	Ruta	Requisito
GET	/notifications · /notifications/unread-count	Autenticado. ?unread_only=true.
POST	/notifications/{id}/read · /notifications/read-all	Autenticado, sobre las propias.
GET · PUT	/admin/notification-settings	Rol global ADMIN. Máximo 5 valores en reminder_days_before, cada uno entre 0 y 30 y sin repetidos; send_hour entre 0 y 23.
GET	/lesson-modules	Autenticado. Solo publicados, salvo para ADMIN y LESSON_EDITOR. Devuelve el árbol completo en una sola petición.

POST · PATCH · DELETE	/lesson-modules[/{id}] · /publish · /reorder	ADMIN o LESSON_EDITOR. El DELETE exige ?confirm=true y devuelve 409 sin él, indicando cuántas lecciones se perderían (RN-36).
GET · POST · PATCH · DELETE	/lessons[/{id}] · /lessons/{id}/resources · /resources/{id}	Lectura para cualquier autenticado si está publicada; escritura solo ADMIN o LESSON_EDITOR.

Contador de no leídas: el frontend lo consulta cada 60 segundos mientras la pestaña esté visible. No se usan WebSockets ni eventos del servidor: mantener una conexión abierta contra un servicio que se suspende no tiene sentido.

8.6 Endpoints internos y de sistema

Método	Ruta	Autenticación
GET	/health	Pública. Ejecuta SELECT 1; sirve de sonda y de mecanismo para mantener despiertos el servicio y la base de datos.
POST	/internal/jobs/reminders	Cabecera X-Job-Token comparada en tiempo constante. No aparece en el esquema público de OpenAPI.

```
// POST /internal/jobs/reminders → resumen de la ejecución
{ "executed_at": "2026-09-05T12:00:03Z", "due_soon_sent": 7,
  "overdue_sent": 2, "skipped_duplicates": 3, "failures": 0 }
```

8.7 Códigos de error

Código	HTTP	Cuándo
UNAUTHENTICATED	401	Token ausente, vencido o inválido.
FORBIDDEN	403	Autenticado sin el permiso necesario.
NOT_FOUND	404	Recurso inexistente o sin visibilidad para el usuario (incluye el proyecto ajeno).
VALIDATION_ERROR	422	Cuerpo inválido.
INVALID_TRANSITION	409	Cambio de estado no permitido.
SUBMISSION_REQUIRED	409	Falta la entrega para pasar a revisión.
SUBMISSION_ALREADY_PENDING	409	Ya hay una entrega sin revisar.
CANNOT_REVIEW_OWN_SUBMISSION	403	El revisor es responsable de la tarea.
PROJECT_ARCHIVED	409	Escritura sobre un proyecto archivado.
ROLE_NAME_TAKEN	409	Nombre de rol repetido en el proyecto.
ROLE_HAS_MEMBERS	409	Borrado de un rol con miembros asignados.
SYSTEM_ROLE_IMMUTABLE	409	Edición de un rol predeterminado.
LAST_ADMIN	409	La operación dejaría el sistema sin administradores activos.
LAST_PROJECT_ADMIN	409	La operación dejaría el proyecto sin nadie con project.archive.
EMAIL_ALREADY_REGISTERED	409	Invitación a un correo que ya tiene cuenta.
INVITATION_EXPIRED	409	Token de invitación vencido.

INVITATION_ALREADY_USED	409	Token de invitación ya utilizado.
-------------------------	-----	-----------------------------------

8.8 Escenarios de error y borde

ID	Situación	Comportamiento esperado
EB-01	Token de invitación vencido o ya usado	Mensaje claro y sugerencia de pedir un enlace nuevo. Nunca se revela si el correo ya tiene cuenta.
EB-02	Invitar un correo que ya tiene cuenta	409 EMAIL_ALREADY_REGISTERED.
EB-03	Invitación pendiente duplicada	Se reemplaza la anterior y el token viejo queda invalidado.
EB-04	Usuario retirado con tareas en curso	Las tareas se conservan sin ese responsable. Si quedan sin ninguno, se marcan visualmente como «sin asignar».
EB-05	Eliminar un rol con miembros	409 indicando cuántos miembros lo tienen.
EB-06	Modificar algo en un proyecto archivado	409 PROJECT_ARCHIVED.
EB-07	Tarea sin responsables que se vence	No hay a quién notificar como responsable; se avisa a los miembros con <code>task.review</code> .
EB-08	Fallo del proveedor de correo	Se registra, se reintenta hasta 3 veces con espera creciente y la notificación en la aplicación se entrega igual. El fallo nunca bloquea la operación que lo originó.
EB-09	El proceso programado no corre un día	Al día siguiente se envía lo del día en curso. No se reconstruyen los atrasados.
EB-10	Fecha límite en el pasado al crear	Se permite, con una advertencia visible: puede ser una tarea que se registra tarde.
EB-11	Último administrador se degrada o desactiva	409 LAST_ADMIN, operación rechazada.
EB-12	URL de entrega con formato inválido	Error de validación indicando el formato esperado.
EB-13	Backend suspendido (arranque en frío)	Según RNF-02. Nunca un error genérico ni una pantalla en blanco.
EB-14	Token de acceso vencido durante el uso	El frontend renueva de forma transparente. Si el refresco falla, redirige al ingreso conservando la ruta destino.

9. Stack tecnológico y despliegue

9.1 Stack

Capa	Decisión
Frontend	Angular con componentes standalone y signals, Tailwind CSS, Impeccable como capa de diseño. Pruebas con Vitest.
Backend	FastAPI sobre Python 3.12, SQLAlchemy 2.x asíncrono, Alembic, Pydantic v2, asyncpg, Argon2 (argon2-cffi), pytest, ruff para formato y análisis estático.
Base de datos	PostgreSQL gestionado en Supabase. Sin Supabase Auth, sin Storage y sin RLS: solo la base de datos.
Autenticación	Propia, con JWT emitidos por FastAPI.
Correo	Resend, con remitente en <code>send.kairospartners.uk</code> .
Proceso programado	GitHub Actions con cron, invocando un endpoint interno protegido.

9.2 Topología de despliegue

Componente	Servicio	Dominio
Frontend	Cloudflare Pages (o Vercel)	<code>app.kairospartners.uk</code>
Backend	Render, plan gratuito	<code>api.kairospartners.uk</code>
Base de datos	Supabase, plan gratuito	interno
Correo	Resend	remitente en <code>send.kairospartners.uk</code>
Programación	GitHub Actions	—

Registros DNS necesarios: `app` (CNAME a Cloudflare Pages), `api` (CNAME a Render), `send` (los que indique Resend, para SPF y DKIM) y `_dmarc` (TXT con `v=DMARC1; p=none; rua=mailto:...`, que **no** crea Resend y hay que poner a mano).

9.3 Variables de entorno del backend

Ningún secreto vive en el código: todo va por variables de entorno, con su entrada correspondiente en `.env.example`.

```
DATABASE_URL=postgresql+asyncpg://...
JWT_SECRET=...
JWT_ACCESS_TTL_MINUTES=15
JWT_REFRESH_TTL_DAYS=7
RESEND_API_KEY=...
EMAIL_FROM=Semillero ML <notificaciones@send.kairospartners.uk>
FRONTEND_URL=https://app.kairospartners.uk
JOB_TOKEN=...
CORS_ORIGINS=https://app.kairospartners.uk
ENVIRONMENT=production
```

9.4 Migraciones y conexión a Supabase

Las migraciones son de Alembic, una por cambio, con mensaje descriptivo, y se ejecutan con `alembic upgrade head` en el comando de arranque del despliegue de Render. Con un equipo de cuatro personas y despliegues

poco frecuentes es aceptable; deja de serlo si en algún momento corren varias instancias.

Conexión a Supabase. Hay que usar el conector agrupado en el puerto **6543** (modo de transacción), no la conexión directa del 5432: el plan gratuito limita las conexiones directas y el agrupador evita agotarlas. Con `asyncpg` en modo de transacción es obligatorio desactivar la caché de sentencias preparadas: `poolclass=NullPool` y `connect_args={"statement_cache_size": 0}`.

10. Estrategia de pruebas

Nivel	Alcance	Herramienta
Unitarias	Servicios con repositorios simulados. Prioridad: máquina de estados de tareas, resolución de permisos y algoritmo del proceso programado.	pytest
Integración	Endpoints contra PostgreSQL real en contenedor. Se derivan de los escenarios Gherkin de <code>user-stories.md</code> y usan los mismos nombres , para poder rastrearlas.	pytest + httpx
Frontend	Servicios y guardas de ruta.	Vitest
Diseño	Analizador de anti-patrones sobre lo que se haya tocado.	npx impeccable detect

Cobertura obligatoria, sin excepciones

1. **Transiciones de estado de tareas:** cada transición válida e inválida de la tabla 5.3.
2. **Resolución de permisos:** `require_project_permission` con rol de sistema, rol a la medida, no miembro y proyecto archivado.
3. **Idempotencia del proceso programado:** ejecutarlo dos veces no debe generar envíos duplicados.

Son los tres puntos donde un error pasa desapercibido y hace daño real.

11. Plan de implementación

Ocho fases ordenadas por dependencia. Cada una deja el sistema funcionando de punta a punta, para poder probarlo de verdad antes de seguir.

Fase	Contenido	Criterio de cierre
0 — Cimientos	Estructura de carpetas; core/config, core/database, core/exceptions con sus manejadores; conexión a Supabase por el agrupador; Alembic; /health; migración inicial con los 14 permisos y la fila de notification_settings; create_admin. Angular con Tailwind, Impeccable instalado e inicializado, esqueleto de rutas e interceptor de arranque en frío. Supabase, Render, Cloudflare Pages, dominio y verificación en Resend.	El frontend desplegado consulta /health del backend desplegado y muestra el resultado.
1 — Acceso HU-01 a HU-03 · RF-01 a RF-12	Modelos de usuarios, invitaciones y tokens; Argon2, JWT y rotación del refresco; endpoints de /auth, /users e /invitations; Resend con plantillas; pantallas de ingreso, invitación, recuperación y perfil; administración de usuarios; guardas de ruta y renovación transparente.	Un administrador invita a alguien, esa persona recibe el correo, crea su cuenta y entra. Recuperar contraseña funciona.
2 — Proyectos, roles y permisos HU-04 a HU-06 · RF-13 a RF-24	Modelos de proyectos, roles, permisos de rol y miembros; creación de proyecto con los tres roles predeterminados en una transacción; require_project_permission con la comprobación de archivado incluida; gestión de miembros y roles a la medida; pantallas de lista, detalle, miembros y editor de roles.	Un administrador crea un proyecto, el líder agrega miembros, crea un rol a la medida y se comprueba que un usuario con ese rol solo puede hacer lo que le corresponde. Es la fase más delicada y no se paraleliza.
3 — Tareas HU-07, HU-08 · RF-25 a RF-32, RF-36, RF-37	Modelos de tareas y responsables; máquina de estados con validación de transiciones; endpoints de tareas y asignación; tablero Kanban con arrastrar y soltar en escritorio y lista con selector en móvil; vista «Mis tareas»; filtros.	El ciclo completo de una tarea funciona salvo la revisión.
4 — Entregas y revisión HU-09 · RF-31, RF-33 a RF-35	Modelos de entregas y comentarios; registro de entrega con validación de la URL de GitHub; aprobación y devolución con la prohibición de autoaprobarse; historial de entregas e hilo de comentarios.	Todos los escenarios de HU-09 pasan, incluidos los de error.
5 — Notificaciones HU-10, HU-11 · RF-38 a RF-45	Modelos de notificaciones y despachos; avisos inmediatos por asignación y por revisión; jobs/reminders.py con la idempotencia; endpoint interno con X-Job-Token; dos flujos de GitHub Actions; campana con contador; pantalla de configuración global.	El proceso se ejecuta dos veces seguidas y no duplica ningún correo. Es donde más errores sutiles aparecen.
6 — Lecciones HU-12 · RF-46 a RF-52	Modelos de módulos, lecciones y recursos; endpoints con la restricción de rol global; catálogo público y editor con reordenamiento.	El editor publica un módulo con lecciones y recursos y los miembros lo ven. Módulo independiente: se puede desarrollar en paralelo a las fases 3 a 5.

7 — Archivado y pulido HU-13, HU-14 · RF-18, RNF-01, RNF-02, RNF-09	Archivar y desarchivar con la restricción de solo lectura verificada endpoint por endpoint; repaso de adaptación a dispositivos, estados vacíos, errores y accesibilidad con Impeccable; verificación del arranque en frío.	La aplicación es usable en un teléfono de 360 px y el arranque en frío no produce ni una pantalla en blanco.
--	---	--

Reparto sugerido para cuatro personas

Fase	Reparto
0	Todos; es corta.
1	Dos en backend, dos en frontend.
2	Dos en backend (roles y permisos), dos en frontend. Sin paralelizar el diseño del sistema de permisos.
3 y 6	Dos personas en tareas, dos en lecciones, en paralelo.
4	Dos personas.
5	Dos personas, una dedicada al trabajo programado.
7	Todos.

12. Fuera de alcance

Explícitamente **no** se construye en la versión 1. Implementar cualquiera de estos puntos sin una decisión expresa es salirse del alcance acordado:

- Tareas recurrentes generadas automáticamente: la periodicidad es solo una etiqueta.
- Carga y almacenamiento de archivos.
- Bitácora de auditoría completa.
- Progreso o certificaciones de lecciones.
- Sprints, estimaciones, puntos de historia y gráficos de avance.
- Integración bidireccional con GitHub (webhooks, estado de commits).
- Notificaciones push o por WhatsApp.
- Multi-semillero o multi-institución.
- Modo sin conexión.
- Renderizado de notebooks o PDF dentro de la aplicación.
- Verificación de correo en el registro: el enlace de invitación ya prueba control de la cuenta.

13. Riesgos y mitigaciones

Riesgo	Mitigación
El arranque en frío de Render arruina la experiencia	Ping programado en horario hábil e indicador de carga honesto (RNF-02).
Supabase pausa el proyecto por inactividad	El mismo ping golpea la base de datos (RNF-07).
El cron de GitHub Actions se retrasa o se salta	Aceptable para avisos diarios; RN-29 lo contempla y el trabajo es idempotente.
Los correos caen en la carpeta de no deseados	Verificar el dominio, publicar DMARC y calentar el envío con volumen bajo.
El sistema de permisos se complica de más	El catálogo es cerrado: catorce permisos, ni uno más sin decisión explícita.
Se acaba el semestre sin terminar	Las fases 0 a 4 ya constituyen un producto útil; las fases 5 a 7 son incrementos postergables.

14. Decisiones de arquitectura registradas

Decisión	Alternativa descartada	Motivo
Autenticación propia con JWT	Supabase Auth	Control total; el equipo quiere aprender el mecanismo; evita atar el modelo de usuarios a un proveedor.
Supabase solo como PostgreSQL	Supabase con RLS y cliente directo	Con toda la autorización en FastAPI, RLS duplicaría las reglas en dos lugares y sería fuente de contradicciones.
Permisos consultados por petición	Permisos dentro del JWT	Los cambios de rol deben surtir efecto de inmediato (RN-22).
GitHub Actions como programador	Render Cron, APScheduler en el proceso	Render Cron es de pago; APScheduler no funciona en un servicio que se suspende.
Idempotencia por restricción de unicidad	Comprobación previa en Python	Cualquier comprobación previa deja una ventana de carrera; la base de datos no.
Periodicidad como etiqueta	Recurrencia real con generación automática	El equipo la pidió así; la recurrencia trae plantillas, instancias y sincronización, y no resuelve un problema actual.
Un rol por usuario por proyecto	Varios roles acumulables	La suma de permisos de varios roles vuelve imposible saber por qué alguien puede algo.
404 en lugar de 403 para no miembros	403 explícito	Un 403 confirmaría la existencia del proyecto.
Sin bitácora de auditoría	Bitácora completa	Descartada explícitamente; <code>created_by</code> y <code>updated_at</code> bastan (RNF-11).
Angular con signals, sin NgRx	NgRx	Desproporcionado para esta escala.
Token de refresco en <code>localStorage</code>	Cookie <code>HttpOnly</code> con CSRF	Herramienta interna sin datos sensibles; se registra como decisión consciente y no como descuido.

15. Matriz de trazabilidad

15.1 Historias de usuario

ID	Historia	Requisitos que cubre
HU-01	Invitar a un nuevo integrante	RF-01, RF-02, RF-04
HU-02	Aceptar la invitación	RF-03, EB-01
HU-03	Recuperar la contraseña	RF-07, RN-41, RN-42
HU-04	Crear un proyecto	RF-13, RF-14, RF-20, RN-13
HU-05	Crear un rol a la medida	RF-21 a RF-24, RN-05, RN-19, RN-22
HU-06	Aislamiento entre proyectos	RN-04
HU-07	Crear y asignar una tarea	RF-25, RF-26, RF-27, RF-39
HU-08	Trabajar y entregar	RF-28, RF-30, RF-31, RF-32, RN-10
HU-09	Revisar una entrega	RF-33, RF-34, RF-42, RN-07, RN-09
HU-10	Recibir recordatorios de vencimiento	RF-38, RF-40, RF-41, RF-44, RF-45, RN-27, RN-28
HU-11	Configurar los recordatorios	RF-43, RN-26
HU-12	Publicar material de estudio	RF-46 a RF-52, RN-31, RN-33
HU-13	Archivar un proyecto terminado	RF-18, RN-15, RN-16, EB-06
HU-14	Usar la plataforma desde el teléfono	RNF-01, RNF-02

15.2 Requisito → historia → fase

Requisitos	Historia	Fase	Verificación principal
RF-01, RF-02, RF-04	HU-01	1	Integración: invitación creada, correo enviado, invite_url devuelta una sola vez.
RF-03	HU-02	1	Integración: token de un solo uso, contraseña mínima de 10 caracteres.
RF-05, RF-06, RF-08	—	1	Integración: rotación del refresco y revocación al cambiar contraseña.
RF-07	HU-03	1	Integración: respuesta 202 idéntica exista o no la cuenta.
RF-09 a RF-12	—	1	Integración: LAST_ADMIN y desactivación sin pérdida de historial.
RF-13, RF-14, RF-20	HU-04	2	Integración: tres roles del sistema y catorce permisos en el rol Líder, todo en una transacción.
RF-15 a RF-17, RF-19	—	2	Integración: LAST_PROJECT_ADMIN al retirar o degradar al último líder.
RF-21 a RF-24	HU-05	2	Integración: ROLE_NAME_TAKEN, SYSTEM_ROLE_IMMUTABLE, ROLE_HAS_MEMBERS, efecto inmediato del cambio de permisos.

RN-04	HU-06	2	Integración: 403 en el proyecto ajeno donde solo se es colaborador; 404 en el proyecto del que no se es miembro.
RF-25 a RF-27, RF-36, RF-37	HU-07	3	Unitarias y de integración: filtros del tablero y vista «Mis tareas».
RF-28 a RF-32	HU-08	3	Cobertura obligatoria: cada transición válida e inválida de la tabla 5.3.
RF-33 a RF-35	HU-09	4	Integración: autoaprobación rechazada, devolución sin comentario rechazada, historial completo.
RF-38 a RF-42, RF-44, RF-45	HU-10	5	Cobertura obligatoria: ejecutar el proceso dos veces sin duplicar envíos.
RF-43	HU-11	5	Integración: validación de <code>reminder_days_before</code> y <code>send_hour</code> ; solo ADMIN.
RF-46 a RF-52	HU-12	6	Integración: borrador invisible, módulo en borrador oculta sus lecciones publicadas, borrado con confirmación.
RF-18	HU-13	7	Cobertura obligatoria: <code>require_project_permission</code> con proyecto archivado.
RNF-01, RNF-02, RNF-09	HU-14	0 y 7	Manual y Vitest: 360 px sin arrastrar y soltar, aviso de arranque en frío a los 3 s.

Anexo A. Discrepancias detectadas en la documentación de origen

Al consolidar los siete documentos aparecieron tres puntos en los que no coinciden entre sí. Se registran aquí en lugar de resolverse en silencio; cada uno necesita una decisión explícita del equipo antes de escribir el código correspondiente.

A.1 Ubicación de `CLAUDE.md`

Qué pasa. El propio `CLAUDE.md`, en su sección «Estructura del repositorio», se ubica en la raíz del proyecto, pero el archivo está en `docs/CLAUDE.md`.

Por qué importa. Las convenciones que contiene son obligatorias para todo el repositorio y se cargan automáticamente solo desde la raíz.

Propuesta. Mover el archivo a `KAIROS/CLAUDE.md`, tal como este documento lo refleja en la figura 5.

A.2 Destinatarios del aviso de tarea vencida

Qué pasa. RF-41 y EB-07 dicen que el aviso de tarea vencida va a los responsables «y al líder del proyecto». RN-25 y el algoritmo de `architecture.md` dicen que va a los responsables y a **los miembros con `task.review`**.

Por qué importa. No son el mismo conjunto: un rol a la medida como «Revisor» tiene `task.review` sin ser líder, y un rol podría no tener `task.review`.

Resolución aplicada. Por la regla de precedencia manda `business-rules.md`: los destinatarios son los responsables activos más los miembros con `task.review`. Este documento ya redacta RF-41 y EB-07 en ese sentido; falta actualizar el PRD.

A.3 Notificaciones sin tipo en el enumerado

Qué pasa. RN-24 afirma que *toda* notificación se entrega por dos canales, y RN-25 lista entre los eventos la invitación a la plataforma y el restablecimiento de contraseña. Pero `notification_kind` solo define cinco valores, todos de tarea o entrega, y en ninguno de esos dos casos puede existir un registro dentro de la aplicación: en el primero la cuenta todavía no existe y en el segundo el usuario no ha iniciado sesión.

Por qué importa. Tal como está, el modelo de datos no admite lo que la regla promete.

Propuesta. Declarar explícitamente en RN-24 que los correos transaccionales de acceso (invitación y restablecimiento) son **solo correo** y quedan fuera del registro de notificaciones, dejando el enumerado como está. Es más simple que agregar dos tipos que nunca se mostrarían en la campana.